技术报告

姓名：邢景丘

说明

本报告只记录我实际动手完成的题目：基础阶段 Linux 方向的 4 个关卡，以及进阶阶段 C/C++ 方向的 4 个关卡。每关按"题目要什么 → 我怎么想 → 怎么做的 → 怎么验证的 → 有什么可改进"来写，重点放在思路和踩坑上，命令只是结果。

一、总体思路

这个平台的题目，本质上都是同一件事：**读懂规则 → 用命令或代码把规则实现出来 → 用样例验证 → 提交**。难点几乎不在算法，而在于两处：一是能不能把题目的要求读准（输入、输出、退出码），二是出错之后能不能自己定位。

所以我给自己定了一个固定流程：

1. 先看 README 和 `--help`，把输入、输出、退出码三件事抄下来；
2. 用最小的例子手算一遍，先知道"正确答案长什么样"；
3. 再动手写；
4. 出错了先读报错原文，不靠猜；
5. 本地验证通过再提交。

后面每一关都是这套流程，只是细节不同。

二、基础阶段（Linux 命令行）

第 1 关 找到入口

题目要求：在工作区里找到校验程序，弄清它怎么用，然后拿到结果。

做法：

- 在 `/workspace` 下列目录，看到 `exam`（可执行文件）、`input.bin`、`cabinet/`、`logs/` 几个东西。
- 跑 `./exam --help`，它把用法全告诉我了：`exam [-h] -e 密钥`；读 `/workspace/input`、写 `/workspace/output`；日志在 `/var/log/ncuscc/exam.log`；退出码 0 表示成功、1 表示处理失

败、2 表示参数错误。

- 按提示去 `cabinet/key.txt` 找到密钥 `MOON`。
- 最后：`cp input.bin input` → `./exam -e MOON` → `cp output flag`。

想法：一开始我以为这关是"破解程序"，其实完全不是——程序自己就老老实实写着怎么用，密钥也放在明面上。这关真正考的是"会不会看说明书"，以及知不知道 `--help` 这个东西的存在。

第 2 关 两段信号

题目要求：信号被拆成两段，分别藏在不同目录里，要找回并拼起来。

做法：

- 在 `catalog/words.txt` 里翻到 `ACCESS_KEY=STAR`，这就是密钥。
- 两段信号分别在 `inbox/signal-B.bin` 和 `archive/2026/signal-A.bin`（注意 A 在子目录里）。
- 两段各自 `cp` 成 `input`、跑一次 exam，输出分别另存为 `AAA` 和 `BBB`。
- 最后 `cat AAA BBB > flag`，按 A 前 B 后的顺序拼起来。

踩的坑：我一开始写的是 `cp /archive/2026/signal-A.bin input`，直接报 `No such file or directory`。原因是开头那个 `/` 表示"从根目录开始找"，而这个 archive 其实在 `/workspace` 底下。去掉开头的斜杠才对。

这件事让我真正分清了相对路径和绝对路径：`archive` 是"我现在所在目录下的 archive"，`/archive` 是"根目录下的 archive"，两者根本不是同一个东西。

第 3 关 读日志并修复输入

题目要求：程序报错了，要读懂报错、把输入修成它能接受的样子。

做法：

- `cat input.txt` 看到内容是 `REPAIR THIS SIGNAL`，`cat key.txt` 得到密钥 `NOVA`（4 个字节）。
- 直接跑，程序报：输入 19 字节，不是密钥 4 字节的整数倍。
- 用 `od -c input` 看原始字节，发现末尾多了一个换行符 `\n`，所以 `REPAIR THIS SIGNAL` 不是 18 字节而是 19 字节。
- `truncate -s -1 input` 把最后的换行砍掉，变成 18 字节；
- `printf ' \n' >> input` 补上一个空格和一个换行，凑成 20 字节（4 的倍数）；

- 再跑一次成功，`cp output flag`。

踩的坑：中间有一次我用 `printf ' ' >> input`，字节数同样是 20，程序却还是不对。因为 `>>` 只能往文件**最末尾**追加，那个空格被加到了换行的后面，位置错了。

这件事让我记住了一句话：**字节数对，不代表内容对。**要确认得用 `od -c` 或 `xxd` 看真实的字节顺序，不能只看 `wc -c`。

第 4 关 让命令找到路

题目要求：写 `/workspace/route.sh`，根据工具返回的不同状态，写结果文件、返回指定退出码。

关键点：

- 题目要求"按命令名调用工具"，说明不能把 `/opt/recruitment/bin/task-probe` 写死，得先把工具目录挂进 `PATH`：`export PATH="$TASK_TOOL_DIR:$PATH"`。要点是"新目录放在前面，同时保留原来的 PATH"，直接覆盖会把系统命令全弄丢。
- 结果文件路径 `ROUTE_RESULT` 也不能写死，评测时会临时注入，必须从环境变量读。
- 配置从 `job.env` 读进来，`ACCESS_TOKEN` 必须 `export`，否则子进程（工具）看不见这个变量。

退出码映射（本题的评分核心）：

工具退出码	结果文件内容	脚本退出码
0	原样写入工具的 `receipt:...`	0
11	`retry:<ITEM>`	75
23	`rejected:<ITEM>`	65
参数非法 / 缺参	只打印 usage，不碰结果文件	64

实现要点：

- 先校验参数再干活：参数个数不等于 2、MODE 不在 `ok/retry/fatal` 里、ITEM 为空、超过 32 个字符、含字母数字以外的字符——全部 `exit 64`，并且**不生成、不修改结果文件。**
- 结果用 `printf '%s\n'` 写，保证恰好一行、带结尾换行，`>` 覆盖旧内容。
- 缓存工具的退出码：`OUT=$(task-probe "$MODE" "$ITEM")` 之后要紧跟着 `CODE=$?`，否则下一条命令会把 `$?` 冲掉。

验证方式：一条一条测——`ok` 看是否 `receipt:...` 且退出 0；`retry` 看 75；`fatal` 看 65；故意给个 `bad HELLO` 看是否 64 且旧结果还在；不带参数看是否 64。

想法：这是我第一次真正理解 `PATH`——它就是一张"命令去哪儿找"的清单。也第一次意识到，一个程序能交出去的东西不止是输出，**退出码同样是一种接口。**

三、进阶阶段（C/C++ 方向）

第 17 关 读数统计

题目要求：读入 n 和 n 个整数，输出四行：`count`、`sum`、`min`、`max`。

做法：一次读入，边读边更新总和、最小值、最大值。最小值/最大值用第一个数初始化，不用先给个假的初值，省掉 `n=1` 的特殊判断。

细节：

- 总和用 `long long`。这题保证不超 32 位，但养成习惯总没错。
- 输出格式严格是 `count=5` 这种，等号两边不能有空格。
- 读不到数据就直接 `return 0`，不希望因为输入异常崩掉。

第 18 关 设备巡检记录

题目要求：n 行记录 `<type> <status> <hours>`，按 printer、scanner、router、server 的固定顺序输出每类的记录数和总小时，最后一行输出 `busiest <type>`。

做法：

- 用结构体数组 `{名字, 计数, 总时长}`，**按固定顺序初始化。**这样输出顺序天然就是对的，省掉一次排序。
- 每读一行，按名字找到对应结构体，计数加一、时长累加。`status` 读进来但不参与计算，因为题目只统计数量和耗时。
- 挑 busiest 时用 `>` 比较，**不是** `>=`。

想法：这题表面是统计题，实际考的是**平局规则。**如果随手写成 `>=`，样例 2 里 printer 和 server 都是 5 小时，就会输出 server，而正确答案是 printer。规则写得很细，得照着字面实现。

第 19 关 固定容量读数窗口

题目要求：只保留最近 8 个读数，输出 `kept`、`oldest`、`last`、`sum`。但原程序容量用尽后会越界写入，要修。

问题定位：

- 原程序用了一个容量为 8 的数组，但写入下标一路往后涨（8、9、10……），第 9 个数就写到数组外面去了；保留计数也没有上限。
- 所以现象不是"输出错"，而是**直接崩或者被保护机制停掉**，反馈是退出码不对。

修复：

- 写入位置改成 `count % 8`。下标到 8 就绕回 0，像转盘一样，新值自动挤掉旧值，剩下的恰好就是"最近 8 个"。
- 输出时最老的元素不在 0 号位，而在"下一次要写的那个位置"，也就是 `count % 8`。
- 不足 8 个时，从 0 开始、全部保留。

收获：数组越界不一定马上报错，可能跑一段才崩，也可能不崩但结果是错的。所以下标必须自己算清楚，"看着能跑"不等于"写对了"。

第 20 关 多文件统计工具

题目要求：工程被拆成入口、统计、格式化三部分，用 CMake 构建，修好之后能干净构建、输出正确结果。

原工程的两类问题：

1. **构建入口没有包含全部实现文件。**`CMakeLists.txt` 只编了 `main.cpp`，`stats.cpp`、`format.cpp` 没参与编译。用的是函数，但找不到实现，于是链接失败。
2. **格式化函数的声明和定义不一致。**头文件里声明的签名和 `.cpp` 里实现的签名对不上。C++ 会把不同签名当成**两个不同的重载函数**，于是被调用的那个没有实现，同样链接失败。C 语言更直接：同一源文件里声明和定义类型冲突，直接编译报错。

修复：

- `add_executable(main main.cpp stats.cpp format.cpp)`，把所有实现文件都编进去；
- 显式写 `set(CMAKE_CXX_STANDARD 17)` 和 `CMAKE_CXX_STANDARD_REQUIRED ON`，把标准钉死，不依赖编译器的默认值；
- 统一函数签名：头文件写 `std::string formatStats(const Stats& s)`，实现完全一致；

- `sum` 用 `long long`——样例 2 里 5 个 10 亿加起来是 50 亿，已经超过 32 位了，用 int 会溢出成负数。

验证：`rm -rf build` 之后从零构建，`cmake -S . -B build` + `cmake --build build`，必须能生成 `build/main`。因为评测就是这么构建的，本地留个编译好的文件没有意义。

四、通用方法总结

1. **先把规则抄下来**：输入、输出、退出码三条，写下来再动手。第 4 关和第 20 关都是"规则没抄全就写代码"最容易出错。
2. **先手算最小样例**：知道正确答案长什么样，才知道自己错在哪。
3. **读报错原文，不要猜**：程序说"19 字节不是 4 的倍数"，那问题就一定在字节数上，别去改密钥。
4. **验证要看得见**：`od -c` 看真实字节、`echo $?` 看退出码、`diff` 比期望输出。凭感觉"应该对了"是最危险的。
5. **同一类错误只犯一次**：第 2 关的路径斜杠、第 3 关的追加位置、第 4 关的 `$?` 被覆盖，说到底都是同一个毛病——没搞清楚这条命令到底作用在哪里。